



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Paola Celina de Jesús Pérez Avin

N° de Cuenta: 318222262

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26 de abril de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Agregar luz de tipo spotlight para la nave de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.

Ejercicio 2: Crear o instanciar una pecera (traslúcida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.

Ejercicio 3: Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.

Ejercicio 4: Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.

EJERCICIO 1

Primero, declaré dos spotlights diferentes para el helicóptero: uno azul en el índice 3 para cuando avanza, y otro rojo en el índice 4 para cuando retrocede.

```
//Luz DELANTERA del helicóptero (blanca) - índice 3
spotLights[3] = SpotLight(0.0f, 0.0f, 1.0f, //Color azul en RGB
    1.0f, 3.0f, //Intensidades ambiental y difusa
    0.0f, 5.0f, 6.0f, //Posición (se actualizará)
    0.0f, -1.0f, 0.0f, //Dirección hacia abajo
    1.0f, 0.01f, 0.001f, //Atenuación de la luz
    45.0f); //Ángulo del cono

//Luz trasera del helicóptero (roja)
spotLights[4] = SpotLight(1.0f, 0.0f, 0.0f, //Color rojo en RGB
    1.0f, 3.0f, //Intensidades ambiental y difusa
    0.0f, 5.0f, 6.0f, //Posición (se actualizará)
    0.0f, -1.0f, 0.0f, //Dirección hacia abajo
    1.0f, 0.01f, 0.001f, //Atenuación de la luz
    45.0f); //Ángulo del cono
```

Luego, en la función ManejaTeclado, programé la tecla Y para mover el helicóptero hacia adelante y activar una bandera heliMovingForward, y la tecla U para moverlo hacia atrás y activar heliMovingBackward.

```
//Inicializar las variables de movimiento
heliMovingForward = false;
heliMovingBackward = false;
pezSubir = false;
pezBajar = false;
```

```
//Controles del Helicóptero
if (key == GLFW_KEY_Y) // Y para adelante (X negativa)
{
    theWindow->heliX -= 1.0;
    theWindow->heliMovingForward = true;
    theWindow->heliMovingBackward = false;
    theWindow->spotLightCount = 4;
}
if (key == GLFW_KEY_U) // U para atrás (X positiva)
{
    theWindow->heliX += 1.0;
    theWindow->heliMovingForward = false;
    theWindow->heliMovingBackward = true;
    theWindow->spotLightCount = 4;
}
```

EJERCICIO 2

Para este ejercicio, primero definí la geometría de la pecera con normales apuntando hacia adentro, de modo que la iluminación interactúe correctamente desde el interior. La pecera consta de 5 caras (frontal, trasera, izquierda, derecha y fondo) más una cara superior que representa la superficie del agua.

```

void CrearPecera()
{
    //Vértices de las paredes y fondo (vidrio)
    //Normales hacia adentro

    GLfloat verticesVidrio[] = {
        //Cara frontal (normal hacia adentro: -Z)
        -2.0f, -2.0f, 2.0f, 0.0f, 0.0f, 0.0f, 0.0f, -1.0f,
        2.0f, -2.0f, 2.0f, 1.0f, 0.0f, 0.0f, 0.0f, -1.0f,
        2.0f, 3.0f, 2.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
        -2.0f, 3.0f, 2.0f, 0.0f, 1.0f, 0.0f, 0.0f, -1.0f,

        //Cara trasera (normal hacia adentro: +Z)
        -2.0f, -2.0f, -2.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f,
        -2.0f, 3.0f, -2.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f,
        2.0f, 3.0f, -2.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
        2.0f, -2.0f, -2.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f,

        //Cara izquierda (normal hacia adentro: -X)
        -2.0f, -2.0f, -2.0f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,
        -2.0f, -2.0f, 2.0f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,
        -2.0f, 3.0f, 2.0f, 1.0f, 1.0f, -1.0f, 0.0f, 0.0f,
        -2.0f, 3.0f, -2.0f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f,

        //Cara derecha (normal hacia adentro: +X)
        2.0f, -2.0f, 2.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
        2.0f, -2.0f, -2.0f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f,
        2.0f, 3.0f, -2.0f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f,
        2.0f, 3.0f, 2.0f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f,

        //Cara inferior (fondo) (normal hacia adentro: -Y)
        -2.0f, -2.0f, -2.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
        2.0f, -2.0f, -2.0f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
        2.0f, -2.0f, 2.0f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f,
        -2.0f, -2.0f, 2.0f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f,
    };
};

```

```

};

//Vértices de la superficie superior (agua)
//Normales hacia adentro (+Y)

GLfloat verticesAgua[] = {
    //Cara superior (normal hacia adentro: +Y)
    -2.0f, 3.0f, 2.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    2.0f, 3.0f, 2.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    2.0f, 3.0f, -2.0f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f,
    -2.0f, 3.0f, -2.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,
};

unsigned int indicesVidrio[] = {
    0,1,2, 0,2,3, //frontal
    4,5,6, 4,6,7, //trasera
    8,9,10, 8,10,11, //izquierda
    12,13,14, 12,14,15, //derecha
    16,17,18, 16,18,19 //fondo
};

unsigned int indicesAgua[] = {
    0,1,2, 0,2,3 //superficie superior
};

```

Para el movimiento del pez en diagonal, usé las teclas I (subir) y K (bajar):

```

//Mover pez con teclas I y K (movimiento diagonal)
float movimiento = velocidadPez * deltaTime;
if (mainWindow.getPezSubir()) {
    posicionPez.y += movimiento;
    posicionPez.z += movimiento * 0.5f; //Diagonal hacia arriba
}
if (mainWindow.getPezBajar()) {
    posicionPez.y -= movimiento;
    posicionPez.z -= movimiento * 0.5f; //Diagonal hacia abajo
}

//Limitar movimiento dentro de la pecera
if (posicionPez.y > 2.0f) posicionPez.y = 2.0f; //Más abajo del agua (Y=3)
if (posicionPez.y < -0.5f) posicionPez.y = -0.5f; //Más arriba del fondo (Y=-2)
if (posicionPez.z > -6.9f) posicionPez.z = -6.9f; //Más atrás de pared frontal (Z=-6)
if (posicionPez.z < -9.1f) posicionPez.z = -9.1f; //Más adelante de pared trasera (Z=-10)
if (posicionPez.x > 9.1f) posicionPez.x = 9.1f; //Más adentro pared derecha (X=10)
if (posicionPez.x < 6.9f) posicionPez.x = 6.9f; //Más adentro pared izquierda (X=6)

//Actualizar posición de la luz puntual del pez (sigue al pez)
pointLights[2].SetPos(glm::vec3(posicionPez.x, posicionPez.y + 0.3f, posicionPez.z));

```

Además, limité el movimiento dentro de los límites de la pecera y apliqué blending para lograr la transparencia del vidrio y el efecto del agua.

```

//Actualizar posición de la luz puntual del pez (sigue al pez)
pointLights[2].SetPos(glm::vec3(posicionPez.x, posicionPez.y + 0.3f, posicionPez.z));

//***** RENDERIZAR PECERA *****
modelPecera = glm::mat4(1.0f);
modelPecera = glm::translate(modelPecera, glm::vec3(8.0f, 0.0f, -8.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPecera));

//Blending para transparencia del vidrio
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE); //Mejora la transparencia

//Paredes de vidrio
texturaVidrio.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
peceraVidrio->RenderMesh();

//Superficie del agua
texturaAgua.UseTexture();
peceraAgua->RenderMesh();

glDepthMask(GL_TRUE);
glDisable(GL_BLEND);

//***** RENDERIZAR PEZ *****
// Cambiar el color del pez (azul abisal)
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.5f, 0.8f))); //Azul abisal

modelPez = glm::mat4(1.0f);
modelPez = glm::translate(modelPez, posicionPez);
modelPez = glm::scale(modelPez, glm::vec3(0.08f, 0.08f, 0.08f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPez));
pezModel.RenderModel();

```

EJERCICIO 3

Declaré una luz puntual azul para el pez en el índice 2 de pointLights:

```
// Luz puntual AZUL para el pez (índice 2)
pointLights[2] = PointLight(0.0f, 0.0f, 1.0f,    //Color azul en RGB
    2.0f, 5.0f,    //Intensidades ambiental y difusa
    0.0f, 1.5f, -8.0f,    //Posición (se actualizará)
    0.5f, 0.3f, 0.1f);    //Atenuación de la luz
pointLightCount++;
```

La lámpara ya existía como luz puntual (índice 1). Para controlar ambas independientemente:

- Tecla L: enciende/apaga la lámpara.
- Tecla P: enciende/apaga la luz del pez.

```
//Apagar/encender lámpara con tecla L
if (mainWindow.getKeys()[GLFW_KEY_L] && !lKeyPressed) {
    lamparaEncendida = !lamparaEncendida;
    lKeyPressed = true;

    if (lamparaEncendida) {
        if (luzPezEncendida) {
            pointLightCount = basePointLightCount + 1; //Lámpara + luz pez
        }
        else {
            pointLightCount = basePointLightCount; //Solo lámpara
        }
        printf("LAMPARA ENCENDIDA!\n");
    }
    else {
        if (luzPezEncendida) {
            pointLightCount = 2; //Luz roja (0) + luz pez (2)
        }
        else {
            pointLightCount = 1; //Solo luz roja
        }
        printf("LAMPARA APAGADA!\n");
    }
}
if (!mainWindow.getKeys()[GLFW_KEY_L]) {
    lKeyPressed = false;
}
```

```

// Apagar/encender luz del pez con tecla P
if (mainWindow.getKeys()[GLFW_KEY_P] && !lPezKeyPressed) {
    luzPezEncendida = !luzPezEncendida;
    lPezKeyPressed = true;

    if (luzPezEncendida) {
        if (lamparaEncendida) {
            pointLightCount = basePointLightCount + 1;
        }
        else {
            pointLightCount = 2; //Luz roja + luz pez
        }
        printf("LUZ DEL PEZ ENCENDIDA\n");
    }
    else {
        if (lamparaEncendida) {
            pointLightCount = basePointLightCount;
        }
        else {
            pointLightCount = 1; //Solo luz roja
        }
        printf("LUZ DEL PEZ APAGADA\n");
    }
}

if (!mainWindow.getKeys()[GLFW_KEY_P]) {
    lPezKeyPressed = false;
}

//Control de spotlight del pez con tecla S
if (mainWindow.getKeys()[GLFW_KEY_S] && !sKeyPressed) {
    spotPezEncendido = !spotPezEncendido;
    sKeyPressed = true;
    printf("SPOTLIGHT DEL PEZ %s\n", spotPezEncendido ? "ENCENDIDO" : "APAGADO");
}

if (!mainWindow.getKeys()[GLFW_KEY_S]) {
    sKeyPressed = false;
}

```

EJERCICIO 4

Implementé un spotlight color naranja (1.0f, 0.5f, 0.0f) en el índice 5 de spotLights:

```

//Luz DELANTERA del helicóptero (blanca)
spotLights[3] = SpotLight(0.0f, 0.0f, 1.0f, //Color azul en RGB
    1.0f, 3.0f, //Intensidades ambiental y difusa
    0.0f, 5.0f, 6.0f, //Posición (se actualizará)
    0.0f, -1.0f, 0.0f, //Dirección hacia abajo
    1.0f, 0.01f, 0.001f, //Atenuación de la luz
    45.0f); //Ángulo del cono

//Luz trasera del helicóptero (roja)
spotLights[4] = SpotLight(1.0f, 0.0f, 0.0f, //Color rojo en RGB
    1.0f, 3.0f, //Intensidades ambiental y difusa
    0.0f, 5.0f, 6.0f, //Posición (se actualizará)
    0.0f, -1.0f, 0.0f, //Dirección hacia abajo
    1.0f, 0.01f, 0.001f, //Atenuación de la luz
    45.0f); //Ángulo del cono

//Spotlight del pez
spotLights[5] = SpotLight(1.0f, 0.5f, 0.0f, //Color naranja en RGB
    1.0f, 5.0f, //Intensidades ambiental y difusa
    0.0f, 1.5f, -8.0f, //Posición (se actualizará)
    1.0f, 0.0f, 0.0f, //Dirección inicial (X positiva)
    1.0f, 0.1f, 0.05f, //Atenuación de la luz
    30.0f); //Ángulo del cono

//Variables para control de dirección del spotlight
glm::vec3 direccionSpotPez = glm::vec3(1.0f, 0.0f, 0.0f);
bool spotPezEncendido = true;
bool sKeyPressed = false;

```

Para controlar su dirección, asigné las teclas:

- Q/E: ajustan la dirección en X.
- R/F: ajustan en Y.
- T/G: ajustan en Z.

```
//Control de dirección del spotlight
//Teclas: Q/E para X, R/F para Y, T/G para Z
if (mainWindow.getKeys()[GLFW_KEY_Q]) {
    direccionSpotPez.x += deltaTime * 2.0f;
    if (direccionSpotPez.x > 1.0f) direccionSpotPez.x = 1.0f;
    printf("Direccion X: %.2f\n", direccionSpotPez.x);
}
if (mainWindow.getKeys()[GLFW_KEY_E]) {
    direccionSpotPez.x -= deltaTime * 2.0f;
    if (direccionSpotPez.x < -1.0f) direccionSpotPez.x = -1.0f;
}
if (mainWindow.getKeys()[GLFW_KEY_R]) {
    direccionSpotPez.y += deltaTime * 2.0f;
    if (direccionSpotPez.y > 1.0f) direccionSpotPez.y = 1.0f;
}
if (mainWindow.getKeys()[GLFW_KEY_F]) {
    direccionSpotPez.y -= deltaTime * 2.0f;
    if (direccionSpotPez.y < -1.0f) direccionSpotPez.y = -1.0f;
}
if (mainWindow.getKeys()[GLFW_KEY_T]) {
    direccionSpotPez.z += deltaTime * 2.0f;
    if (direccionSpotPez.z > 1.0f) direccionSpotPez.z = 1.0f;
}
if (mainWindow.getKeys()[GLFW_KEY_G]) {
    direccionSpotPez.z -= deltaTime * 2.0f;
    if (direccionSpotPez.z < -1.0f) direccionSpotPez.z = -1.0f;
}
```

Además, la tecla S permite encender/apagar este spotlight independientemente.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica.
- c. Conclusión.

Los ejercicios presentaron un nivel de complejidad moderado a alto, ya que requirieron integrar múltiples tipos de luces (direccional, puntual, spotlight) con modelado jerárquico para que las luces siguieran a objetos móviles. El ejercicio 2 implicó además trabajar con normales hacia adentro y transparencias (blending), lo cual añadió dificultad. El ejercicio 3 permitió comprender la gestión dinámica del conteo de luces según el estado de encendido/apagado. El ejercicio 4 extendió el control interactivo a la dirección del haz de luz.

4.- Bibliografía en formato APA.

- *OpenGL - the industry's foundation for high performance graphics*. (2011, July 19). The Khronos Group. <https://www.khronos.org/opengl/>.
- *Documentation*. (n.d.). GLFW. <https://www.glfw.org/documentation.html>.
- *OpenGL Wiki*. (n.d.). https://wikis.khronos.org/opengl/Main_Page.
- *Tutorial 21 - Spot Light*. (n.d.). <https://ogldev.org/www/tutorial21/tutorial21.html>.
- *LearnOpenGL - light casters*. (n.d.). <https://learnopengl.com/lighting/light-casters>.
- *Classical Street Lamp* | *CGTRader*. (n.d.). CGTrader. <https://www.cgtrader.com/items/4581624/download-page>.